

**Lab 1 Report: Data Visualization, Data Preprocessing, and Statistical Analysis Using
Python in Jupyter Notebook**

Anushka Nanaware

MSCS-634-M50 – Advanced Big Data and Data Mining

Dr. Satish Penmatsa

Department of Computer Science

University of the Cumberland

September 2020, 2026

1. Data Collection

For this lab, I picked the **tips dataset**, which is real data from a restaurant showing 244 individual bills, along with the tip amount, day, time, party size, and a couple other details about each customer. I liked this dataset because it's actual sales data, it mixes numbers and categories together, and it has a few naturally large bills that work well later on when we get to outlier detection. I loaded it straight from a public URL into a Pandas DataFrame and checked that it loaded correctly.

```
Anushka Nanaware Course:MSCS-634-M50 – Advanced Big Data and Data Mining Assignment: Lab 1 – Data Visualization, Data Preprocessing, and Statistical Analysis
```

```
[2] import pandas as pd
✓ Os df = pd.read_csv("https://raw.githubusercontent.com/mwaskom/seaborn-data/master/tips.csv")
df.head()
```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

Next steps: [New interactive sheet](#)

2. Data Visualization

Chart 1 — Scatter Plot: Total Bill vs. Tip

There's a pretty clear pattern here — bigger bills come with bigger tips. The correlation between the two is about 0.68, which is a fairly strong connection. That makes sense, since a lot of people tip based on a percentage of the bill instead of just picking a flat number. I did notice that for the really big bills (around \$45–50), the tips get a bit less predictable — some people still tip generously, others don't.

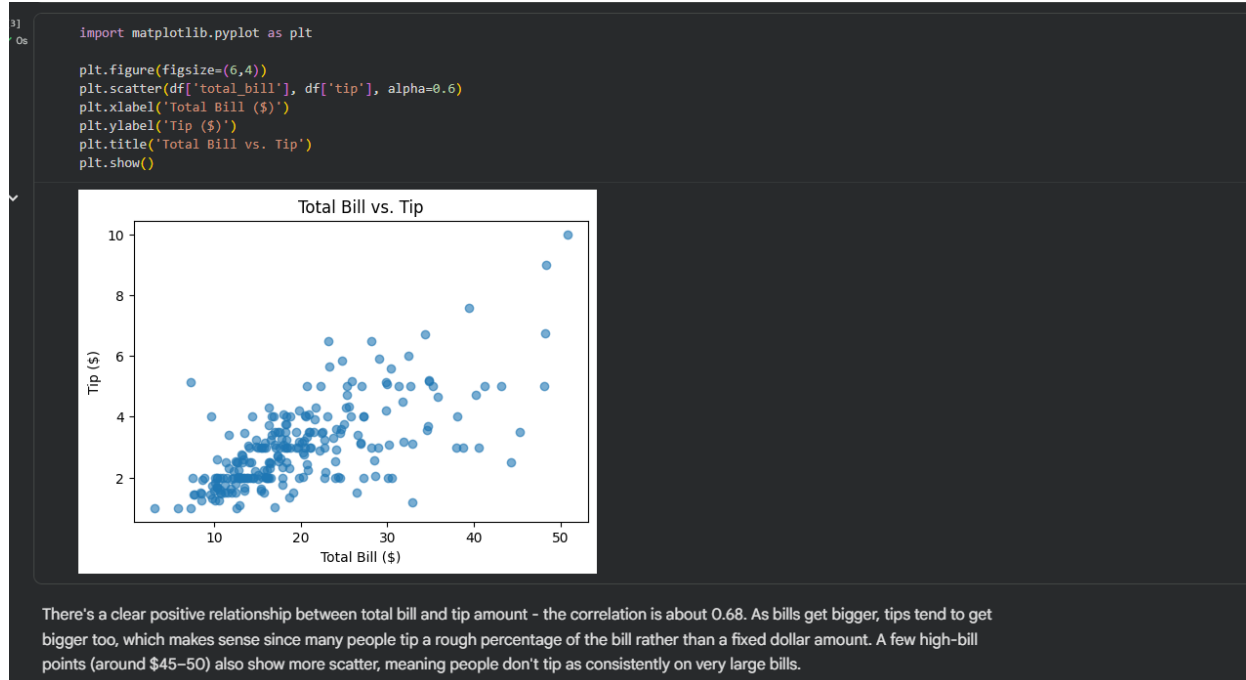


Chart 2 — Box Plot: Total Bill by Day

Weekends clearly bring in more money — Saturday and Sunday both have higher median bills and a wider spread than Thursday and Friday. The weekend days also show more of those little circle outliers above the box, meaning a few really big bills (over \$48) popped up on those days. Thursday and Friday look calmer and more consistent by comparison.



3. Data Preprocessing

3.1 Handling Missing Values

The tips dataset doesn't actually come with any missing data, so I intentionally added some blanks myself (6 in total_bill, 5 in tip, 4 in size) just so I'd have something real to practice cleaning. I used a fixed random seed so the same "random" gaps show up every time I re-run it. Then I fixed them three different ways: filled total_bill gaps with the column average, filled tip gaps by carrying the previous value forward, and filled size gaps with the most common party size. After that, there were zero missing values left, and no rows were lost.

```
[5]
✓ 0s
import numpy as np

np.random.seed(42)
df_missing = df.copy()

total_bill_idx = np.random.choice(df_missing.index, 6, replace=False)
tip_idx = np.random.choice(df_missing.index, 5, replace=False)
size_idx = np.random.choice(df_missing.index, 4, replace=False)

df_missing.loc[total_bill_idx, 'total_bill'] = np.nan
df_missing.loc[tip_idx, 'tip'] = np.nan
df_missing.loc[size_idx, 'size'] = np.nan

# BEFORE: detect and report missing values
print("Missing values per column:")
print(df_missing.isnull().sum())

Missing values per column:
total_bill    6
tip           5
sex           0
smoker        0
day           0
time          0
size          4
dtype: int64

[6]
✓ 0s
# AFTER: handle missing values using a few different techniques
df_clean = df_missing.copy()

# Mean replacement for total bill
df_clean['total_bill'] = df_clean['total_bill'].fillna(df_clean['total_bill'].mean())

# Forward fill for tip
df_clean['tip'] = df_clean['tip'].ffill()

# Mode replacement for size
df_clean['size'] = df_clean['size'].fillna(df_clean['size'].mode()[0])
```

3.2 Outlier Detection and Removal

I used the IQR method on `total_bill` to find outliers. Q1 came out to 13.30, Q3 to 24.02, so the IQR was 10.61. That gave me a lower boundary around -2.50 and an upper boundary around 39.94. Anything above 39.94 counted as an outlier, and 10 bills crossed that line — all of them over \$40, with the biggest one at \$50.81. I removed those, which brought the dataset down from 244 rows to 234.

```
[7] ✓ 0s # IQR method on total_bill
Q1 = df_clean['total_bill'].quantile(0.25)
Q3 = df_clean['total_bill'].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

print(f"Q1 = {Q1:.2f}, Q3 = {Q3:.2f}, IQR = {IQR:.2f}")
print(f"Lower bound = {lower_bound:.2f}, Upper bound = {upper_bound:.2f}")

outliers = df_clean[(df_clean['total_bill'] < lower_bound) | (df_clean['total_bill'] > upper_bound)]
print(f"\nNumber of outliers found: {len(outliers)}")
outliers[['total_bill', 'tip', 'day']]
```

... Q1 = 13.41, Q3 = 24.02, IQR = 10.61
Lower bound = -2.50, Upper bound = 39.94

Number of outliers found: 10

	total_bill	tip	day
59	48.27	6.73	Sat
95	40.17	4.73	Fri
102	44.30	2.50	Sat
142	41.19	5.00	Thur
156	48.17	5.00	Sun
170	50.81	10.00	Sat
182	45.35	3.50	Sun
184	40.55	3.00	Sun
197	43.11	5.00	Thur
212	48.33	9.00	Sat

3.3 Data Reduction

I cut the dataset down in two different ways: first by randomly keeping 70% of the rows, and separately by grabbing a fixed sample of 100 rows. I also dropped the "sex" column entirely, since it's not something we really need for looking at bill and tip patterns.

```
[8] ✓ Os # Remove the outliers
df_no_outliers = df_clean[(df_clean['total_bill'] >= lower_bound) & (df_clean['total_bill'] <= upper_bound)]

print("Shape before removing outliers:", df_clean.shape)
print("Shape after removing outliers:", df_no_outliers.shape)

Shape before removing outliers: (244, 7)
Shape after removing outliers: (234, 7)

[9] ✓ Os print("Original shape:", df_no_outliers.shape)

# Sampling by percentage (70% of rows)
df_sampled_pct = df_no_outliers.sample(frac=0.7, random_state=42)
print("Shape after 70% sampling:", df_sampled_pct.shape)

# Sampling by fixed number (100 rows)
df_sampled_n = df_no_outliers.sample(n=100, random_state=42)
print("Shape after sampling 100 rows:", df_sampled_n.shape)

# Dimension elimination - drop a less relevant column
df_reduced_cols = df_no_outliers.drop(columns=['sex'])
print("Columns before:", list(df_no_outliers.columns))
print("Columns after dropping 'sex':", list(df_reduced_cols.columns))

... Original shape: (234, 7)
Shape after 70% sampling: (164, 7)
Shape after sampling 100 rows: (100, 7)
Columns before: ['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size']
Columns after dropping 'sex': ['total_bill', 'tip', 'smoker', 'day', 'time', 'size']

Insight: I reduced the dataset two ways: row-wise sampling (down to 70% of rows, and separately down to a fixed 100 rows) to demonstrate shrinking dataset size while keeping it representative, and column-wise reduction by dropping sex, since gender isn't especially relevant if our main interest is bill amount and tipping patterns.
```

3.4 Data Scaling and Discretization

I scaled `total_bill` three ways — Min-Max, Z-score, and decimal scaling — just to see how each one reshapes the numbers differently. Then I split `total_bill` into three simple groups: Low (up to \$15), Medium (\$15–25), and High (over \$25). Most of the bills landed in the Medium range (112), followed by Low (78), then High (44).

2

21.010000

Medium

3

23.680000

Medium

4

24.590000

Medium

5

25.290000

High

6

19.822699

Medium

7

26.880000

High

8

15.040000

Medium

9

14.780000

Low

[12]

✓ Os

df_scale["bill_category"].value_counts()

count

bill_category

Medium

112

Low

78

High

44

dtype: int64

Insight: All three scaling methods rescale total_bill differently — Min-Max squeezes everything into 0–1, Z-score centers around 0 in terms of standard deviations, and decimal scaling just shifts the decimal point. For discretization, I split bills into Low (≤ 15), *Medium* (15–25), and High (>\$25) categories — the split came out to 78 Low, 112 Medium, and 44 High, showing most transactions cluster in the mid-range.

Double-click (or enter) to edit

4. Statistical Analysis

4.1 General Overview of the Data

Running `.info()` confirmed 234 rows with nothing missing across all 7 columns — 3 number columns and 4 category columns. `.describe()` showed the average bill sits around \$18.75, the average tip is about \$2.89, and the average table size is 2.5 people.


```
[13]
✓ Os
df_no_outliers.info()

<class 'pandas.core.frame.DataFrame'>
Index: 234 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  234 non-null    float64
1   tip         234 non-null    float64
2   sex         234 non-null    object
3   smoker      234 non-null    object
4   day         234 non-null    object
5   time        234 non-null    object
6   size        234 non-null    float64
dtypes: float64(3), object(4)
memory usage: 14.6+ KB

[14]
✓ Os
df_no_outliers.describe()

   total_bill  tip  size
count  234.000000  234.000000  234.000000
mean    18.745886   2.894402   2.495726
std     7.258822   1.209939   0.899441
min     3.070000   1.000000   1.000000
25%    13.302500   2.000000   2.000000
50%    17.490000   2.750000   2.000000
75%    22.757500   3.500000   3.000000
max    39.420000   7.580000   6.000000

Insight: .info() confirms we have 234 rows (after removing outliers) with no missing values across all 7 columns — 3 numeric (total_bill, tip, size) and 4 categorical. .describe() shows the numeric spread: average bill is about 18.75, averagetip about 2.89, and average party size 2.5 people.
```

(Your screenshot Screenshot_2026-09-20_171249.png shows mostly the same discretization results plus the very start of .info() — it overlaps with the two screenshots already placed above, so you don't need to include it separately unless you just want extra proof of your work.)

4.2 Central Tendency (Mean, Median, Mode, etc.)

For total_bill: the lowest bill was \$3.07, the highest was \$39.42, the average was \$18.75, and the middle value (median) was \$17.49. The most common value (mode) was \$19.82 — which is actually kind of funny, because that's the exact average we used earlier to fill in the missing values, so it artificially became the "most repeated" number. For tip: bills ranged from \$1.00 to \$7.58, averaging \$2.89, with a median of \$2.75 and a mode of \$2.00.

```
[15] ✓ Os
print("=== total_bill ===")
print("Min:", df_no_outliers['total_bill'].min())
print("Max:", df_no_outliers['total_bill'].max())
print("Mean:", round(df_no_outliers['total_bill'].mean(), 2))
print("Median:", df_no_outliers['total_bill'].median())
print("Mode:", df_no_outliers['total_bill'].mode()[0])

=== total_bill ===
Min: 3.07
Max: 39.42
Mean: 18.75
Median: 17.490000000000002
Mode: 19.822899159663866

[16] ✓ Os
print("=== tip ===")
print("Min:", df_no_outliers['tip'].min())
print("Max:", df_no_outliers['tip'].max())
print("Mean:", round(df_no_outliers['tip'].mean(), 2))
print("Median:", df_no_outliers['tip'].median())
print("Mode:", df_no_outliers['tip'].mode()[0])

=== tip ===
Min: 1.0
Max: 7.58
Mean: 2.89
Median: 2.75
Mode: 2.0
```

Insight: For total_bill: min 3.07, *max* 39.42, mean 18.75, *median* 17.49. The mean sits a bit above the median, meaning the distribution still leans slightly right even after removing the biggest outliers. For tip: values range from 1.00 to 7.58, with a mean of 2.89 and *median* of 2.75, showing a fairly tight, only slightly skewed distribution.

4.3 Spread of the Data (Range, IQR, Variance, Standard Deviation)

Total_bill has a range of \$36.35 (from \$3.07 up to \$39.42). Q1 sits at \$13.30 and Q3 at \$22.76, giving an IQR of \$9.46. The variance came out to about 52.69, and the standard deviation was \$7.26 — a reasonably moderate spread, especially now that the big outliers are already gone.

```
[17] ✓ Os
range_val = df_no_outliers['total_bill'].max() - df_no_outliers['total_bill'].min()
q1 = df_no_outliers['total_bill'].quantile(0.25)
q3 = df_no_outliers['total_bill'].quantile(0.75)
iqr = q3 - q1
variance = df_no_outliers['total_bill'].var()
std_dev = df_no_outliers['total_bill'].std()

print("Range:", round(range_val, 2))
print("Q1:", q1, " Q3:", q3, " IQR:", round(iqr, 2))
print("Variance:", round(variance, 2))
print("Standard Deviation:", round(std_dev, 2))

Range: 36.35
Q1: 13.302499999999998 Q3: 22.7575 IQR: 9.46
Variance: 52.69
Standard Deviation: 7.26
```

Insight: total_bill has a range of 36.35 (from 3.07 to 39.42), with Q1 = 13.30 and Q3 = 22.76, giving an IQR of 9.46 — this tells us the middle 50% of bills span a fairly moderate 9.46 window. The standard deviation of 7.26 (variance = 52.69) shows a reasonable, not extreme, amount of spread now that we've already removed the outliers in Step 3.

4.4 Correlation Between Variables

Total_bill and tip correlate at 0.618, which is a decent positive relationship — a bit lower than the 0.676 we saw earlier before removing outliers, which tells us those really big bills were

inflating that number a little. Total_bill and size correlate at 0.571, and tip and size at 0.473 — so bigger groups do tend to spend more and tip a bit more, just not as strongly as the bill-and-tip connection itself.

```
[18] correlation_matrix = df_no_outliers[['total_bill', 'tip', 'size']].corr()
correlation_matrix
```

	total_bill	tip	size
total_bill	1.000000	0.618023	0.571247
tip	0.618023	1.000000	0.472931
size	0.571247	0.472931	1.000000

Next steps: [New interactive sheet](#)

Insight: total_bill and tip correlate at 0.618 - a moderately strong positive relationship, slightly weaker than the 0.676 we saw back in Step 2 before removing outliers (makes sense, since those large-bill outliers were adding extra pull to that relationship). total_bill and size correlate at 0.571, and tip and size at 0.473 - meaning bigger parties tend to run up bigger bills and slightly bigger tips, though the effect is more moderate than the bill-tip relationship itself.

Challenges and Decisions I Made

- Since the dataset didn't come with any real missing values, I had to add some myself on purpose, just so I could actually practice fixing them.
- I ran into an error using `fillna(method="ffill")` — turns out newer versions of pandas got rid of that option, so I switched to using `.ffill()` directly instead.
- I only checked for outliers in total_bill, since it was the column with the clearest spread and the most obvious extreme values.
- For splitting bills into Low/Medium/High, I picked \$15 and \$25 as the cutoffs based on roughly where the data's quartiles landed, instead of just picking round numbers arbitrarily